

Laboratorio Experiencia Físico-Digital Piloto de nave/dron

Andrés Botero, Jade Moreno, Sofía Reina and Vivi Delgado

Resumen—En este trabajo se desarrolló una experiencia físico-digital mediante la integración de un microcontrolador Arduino Pro Micro con una aplicación interactiva desarrollada en Godot. Para la construcción del controlador físico se utilizó una placa perforada, sobre la cual se ensamblaron un joystick analógico y tres pulsadores. Las señales generadas por estos componentes fueron capturadas por el microcontrolador y transmitidas hacia el entorno digital mediante una conexión USB. Los datos recibidos fueron procesados en Godot y utilizados para controlar una experiencia interactiva de temática espacial, en la cual el usuario maneja una nave e interactúa con diferentes asteroides. La comunicación implementada utiliza mensajes con un tamaño inferior a 20 bytes, por lo que su impacto sobre el rendimiento es reducido. Asimismo, se realizó una validación básica de los mensajes, verificando que cumplieran con el formato establecido y descartando aquellos que no lo hicieran. Los resultados obtenidos durante las pruebas permitieron comprobar el funcionamiento estable de la comunicación entre el controlador físico y la aplicación, demostrando la viabilidad del uso de Arduino y USB como medio de interacción para experiencias digitales desarrolladas en Godot.

Index Terms—Arduino Pro Micro, Godot, comunicación USB, interacción físico-digital, joystick analógico, microcontrolador, transmisión de datos.

I. INTRODUCCIÓN

LA integración entre dispositivos físicos y entornos digitales permite desarrollar experiencias interactivas en las que las acciones realizadas mediante un controlador físico pueden ser interpretadas y representadas dentro de una aplicación. En este contexto, los microcontroladores permiten adquirir señales provenientes de diferentes dispositivos de entrada y transmitirlas hacia un entorno de software, estableciendo una comunicación entre el hardware y la experiencia digital.

En el presente laboratorio se desarrolló una experiencia físico-digital mediante la integración de un microcontrolador Arduino Pro Micro con una aplicación interactiva desarrollada en Godot. Para la construcción del controlador físico se utilizó una placa perforada, sobre la cual se ensamblaron los diferentes componentes de entrada. El sistema incorpora un joystick analógico y tres pulsadores, cuyos valores y estados son capturados por el microcontrolador y transmitidos hacia el entorno digital mediante una conexión USB.

Como aplicación de esta integración se desarrolló un juego de temática espacial en el que el usuario controla una nave e interactúa con diferentes asteroides presentes en el entorno. Las señales generadas por el joystick y los pulsadores permiten controlar las acciones definidas dentro del juego, estableciendo una relación directa entre las entradas físicas y el comportamiento de los elementos digitales.

Para lograr esta comunicación fue necesario establecer una estructura para los mensajes enviados desde el Arduino Pro Micro hacia Godot y realizar una validación básica de los datos recibidos. Los mensajes utilizados durante las pruebas presentan un tamaño inferior a 20 bytes, por lo que su impacto sobre el rendimiento de la comunicación es reducido. Debido a que la transmisión de información se realiza de manera continua, los mensajes que no cumplen con el formato establecido pueden ser descartados, permitiendo que los siguientes datos recibidos mantengan el funcionamiento de la interacción.

La utilización de USB como medio de comunicación permite simplificar la integración entre el controlador físico y la aplicación, evitando configuraciones adicionales o dependencias externas para establecer la comunicación. De esta manera, el desarrollo permite aplicar conceptos relacionados con la adquisición de señales, transmisión de datos y comunicación entre hardware y software en una experiencia interactiva desarrollada en Godot.

II. OBJETIVOS

II-A. Objetivo general

Implementar y evaluar una experiencia físico-digital basada en la comunicación entre un microcontrolador Arduino Pro Micro y un entorno interactivo en Godot, utilizando los protocolos USB HID y Teleprot para analizar el flujo de información y el control en tiempo real de un simulador de vuelo.

II-B. Objetivos específicos

- Diseñar y construir un circuito físico basado en un microcontrolador Arduino Pro Micro, un joystick analógico y tres pulsadores para la captura de señales análogas y digitales.
- Investigar y aplicar los fundamentos técnicos de la comunicación USB nativa (HID) y protocolos seriales ligeros como Teleprot para la transmisión de comandos hacia entornos virtuales.
- Desarrollar la integración de las señales físicas con el simulador interactivo “Piloto de nave/dron” en Godot Engine, validando el comportamiento operativo bajo condiciones reales.
- Analizar el rendimiento de la comunicación en términos de tamaño de paquete, eficiencia del bus y estrategias de tolerancia a fallos mediante validación de formato.

III. MARCO TEÓRICO

III-A. Estándares de Construcción de Paquetes de Datos (Framing)

En el contexto de las comunicaciones seriales, el flujo de información transmitida debe estructurarse adecuadamente para que el sistema receptor pueda distinguir de manera inequívoca el principio y el final de cada mensaje [1]. Para lograr este propósito, existen diferentes estrategias de empaquetado o *framing*:

- **Longitud fija frente a estructuración por delimitador:** Un enfoque común es establecer tramas de longitud fija, donde el receptor conoce de antemano exactamente cuántos bytes componen un paquete. Esta técnica minimiza la sobrecarga computacional relacionada con el análisis sintáctico del mensaje, pero resulta ineficiente si el tamaño de los datos varía frecuentemente, ya que obliga a rellenar con bytes vacíos desperdiciando el ancho de banda del canal [2]. Por otro lado, la estructuración por delimitador flexibiliza la longitud del paquete al utilizar caracteres especiales (conocidos como banderas o *flags*) para señalar los bordes de la trama. Esto optimiza el uso del canal de comunicación, aunque exige que el decodificador escanee de forma continua el flujo de entrada en busca de dichos marcadores [1].
- **Uso de encabezados (Headers):** Es una práctica extendida que los paquetes estructurados incluyan un bloque inicial llamado encabezado, cuya función principal es proporcionar metadatos cruciales sobre la carga útil (*payload*). Dicho encabezado suele especificar parámetros como la longitud exacta de los datos adjuntos, el tipo de instrucción contenida o las direcciones lógicas de la red, facilitando así la preparación de buffers en memoria y el procesamiento por las capas superiores antes de que el paquete llegue completo [2].
- **Byte Stuffing (Relleno de bytes y Escaping):** La adopción de delimitadores introduce un desafío técnico crítico en la transmisión de datos crudos: si el byte empleado como bandera delimitadora aparece accidentalmente dentro de la propia carga útil, el sistema receptor cerrará y fragmentará la trama de manera prematura. Para mitigar esta colisión de datos, se emplea la técnica de *byte stuffing* (relleno de bytes) o el uso de secuencias de escape. Este procedimiento consiste en insertar un carácter especial (como un byte nulo o una barra invertida) inmediatamente antes de la coincidencia conflictiva, indicándole al módulo de recepción que debe tratar el siguiente byte puramente como información y no como una instrucción de control [1].

III-B. JSON como Formato de Intercambio y Deserialización en Entornos Interactivos

JSON (JavaScript Object Notation) se ha consolidado como un estándar universal para la transferencia de información entre sistemas gracias a que es un formato de texto plano, altamente ligero y fácilmente interpretable. Al integrar interfaces de hardware físico, como microcontroladores, con entornos simulados, el proceso algorítmico encargado de traducir estas

cadenas de texto continuo en estructuras de datos utilizables se conoce como deserialización. Dado que la construcción de ecosistemas interactivos puede abordarse desde distintos motores gráficos, el enfoque de deserialización varía sustancialmente según la arquitectura del software:

- **Deserialización en C# (Unity):** En el contexto de Unity, el proceso requiere mapear el texto a objetos fuertemente tipados. El motor provee `JsonUtility`, una biblioteca nativa implementada en C++ que garantiza tiempos de procesamiento excepcionalmente rápidos y una asignación de memoria mínima, evitando invocar frecuentemente al recolector de basura (*garbage collector*) durante ciclos de actualización rápidos [3]. No obstante, presenta restricciones al no admitir la deserialización directa de diccionarios o arreglos complejos [3]. Para solventar estas limitaciones, se recurre a `Newtonsoft.Json` (*Json.NET*), una biblioteca de terceros que proporciona una libertad estructural profunda para evaluar datos dinámicos, aunque introduce una mayor sobrecarga computacional [4].
- **Deserialización nativa en Godot Engine:** Como contraparte y herramienta implementada en el desarrollo práctico de este laboratorio, Godot Engine aborda el procesamiento de datos bajo un paradigma de tipado dinámico, especialmente al emplear su lenguaje principal, GDScript. Godot resuelve la deserialización sin depender de bibliotecas externas de terceros mediante su clase global `JSON` [5]. A diferencia de Unity, donde se requiere la declaración previa de clases contenedoras, el método `JSON.parse_string()` de Godot interpreta la cadena serializada y la convierte de manera inmediata en una estructura de datos dinámica, usualmente un objeto de tipo *Dictionary* o *Array* [5]. Si bien esto agiliza el desarrollo iterativo y reduce la verbosidad del código, la ausencia de un esquema rígido tipado transfiere la responsabilidad de la validación de integridad (como la verificación de la existencia de claves específicas) directamente a la lógica del desarrollador [5].

III-C. Métodos de Comprobación de Integridad de Datos

Dado que los canales de comunicación serial están frecuentemente expuestos a ruidos eléctricos e interferencias electromagnéticas, es indispensable implementar métodos matemáticos que permitan al receptor validar si los bytes transferidos sufrieron alteraciones durante el recorrido [1]:

- **Bit de paridad:** Representa el mecanismo de verificación más rudimentario. Se basa en concatenar un bit redundante al final de la palabra de datos con el objetivo de forzar que el conteo total de unos lógicos (1) sea par o impar. Si bien su costo computacional es prácticamente inexistente en microcontroladores, su nivel de robustez es deficiente; este método falla sistemáticamente ante ráfagas de error, ya que no puede alertar anomalías si un número par de bits cambia de estado de manera simultánea [2].
- **Checksum (Suma de comprobación módulo 256 y XOR):** Involucra ejecutar operaciones algebraicas elementales sobre el paquete. Por ejemplo, consiste en reali-

zar la suma (descartando el desbordamiento de 8 bits) de todos los bytes del mensaje, o aplicar la compuerta lógica XOR de forma acumulativa sobre la secuencia, anexando el resultado como firma final de la trama. El dispositivo receptor repite el cálculo para corroborar las firmas. Estas técnicas consumen recursos computacionales muy bajos, haciéndolas ideales para el hardware empujado [2]. Su robustez es considerada de nivel medio, puesto que son vulnerables a errores de transposición (cuando dos bytes intercambian posiciones exactas sin corromperse individualmente) y a la inserción de ceros falsos [1].

- **CRC (Comprobación de Redundancia Cíclica):** Es un enfoque polinomial que ofrece alta precisión. Analiza la totalidad del bloque de bits asumiendo que es un coeficiente polinómico gigante, el cual se divide iterativamente (mediante aritmética XOR) por una constante matemática conocida como polinomio generador; el residuo resultante de esta división se adjunta al mensaje [1]. El CRC destaca por una robustez excepcional, logrando detectar alteraciones de ráfagas largas e inversiones complejas que el *checksum* omitiría [2]. La contrapartida de esta confiabilidad es su costo computacional, el cual es el más alto entre las opciones, y que muchas veces exige el uso de optimizaciones en memoria, como tablas precalculadas (*look-up tables*), para evitar cuellos de botella en la ejecución del firmware [1].

IV. METODOLOGÍA Y DESARROLLO

IV-A. Descripción del Montaje Físico y Hardware

Para el desarrollo de la experiencia físico-digital se implementó un sistema embebido basado en hardware de adquisición de señales compuesto por un microcontrolador **Arduino Pro Micro**, una placa perforada para asegurar la estabilidad de las conexiones y soldaduras, un joystick analógico de dos ejes, tres pulsadores digitales y el cableado necesario para interconectar los componentes. A diferencia del diseño estándar con Arduino Uno y potenciómetro individual, la elección del Arduino Pro Micro se fundamenta en su capacidad nativa de emulación de dispositivos USB (USB On-The-Go y perfil HID), lo que optimiza la transmisión de datos hacia el entorno digital. Los pulsadores y el joystick fueron distribuidos en la placa perforada para capturar tanto entradas digitales discretas (como acciones de disparo, impulso o cambio de modo) como entradas analógicas continuas (empleadas para el control vectorial o de aceleración).

IV-B. Selección del Simulador e Implementación en Godot Engine

Como entorno de simulación interactiva se asignó el perfil **Piloto de nave/dron**, el cual exige un control continuo de la altitud y aceleración mediante los ejes del joystick, complementado por los pulsadores para funciones de disparo, impulso y alternancia de modos. El desarrollo del software interactivo no se realizó sobre el motor Unity predeterminado en la guía, sino que se implementó íntegramente en **Godot Engine**, empleando su arquitectura de nodos y scripts en GDScript para gestionar la recepción de eventos físicos. Todo el código fuente

del sistema, incluyendo los scripts de control y la estructura del proyecto, se encuentra disponible en el repositorio oficial del grupo [7].

IV-C. Protocolos de Comunicación Implementados

La estrategia de comunicación entre el microcontrolador y el simulador en Godot contempló el análisis y desarrollo de las siguientes alternativas:

- **USB HID (Human Interface Device):** Se configuró directamente en el firmware del Arduino Pro Micro para que el microcontrolador se comporte como un periférico de juego estándar. Mediante este mecanismo, el controlador físico puede transmitir las entradas del joystick y los botones como eventos de un dispositivo HID. El firmware define un reporte de tipo gamepad, con campos para los botones y los ejes X y Y, permitiendo que el sistema operativo y el entorno de desarrollo reconozcan el dispositivo como un controlador de entrada. Este enfoque simplifica la integración con Godot, ya que las acciones del controlador pueden asociarse mediante el sistema de entradas (*Input Map*) evitando depender exclusivamente de la interpretación de cadenas de texto para el funcionamiento de la interacción..
- **Teleplot:** Protocolo orientado a la transmisión estructurada de paquetes de telemetría y comandos de control a través del puerto serial, facilitando un flujo bidireccional de datos con marcas de tiempo y control de flujo entre el hardware y el entorno gráfico. En el código generamos mensajes estructurados mediante identificadores para los diferentes estados de entrada: A, B, C y D para los botones, y X y Y para los ejes analógicos. Los mensajes siguen un formato del tipo `¿VARIABLE:VALOR`, acompañado por un salto de línea, lo que permite visualizar gráficamente la variación de las entradas durante la manipulación del controlador físico. De esta manera, Teleplot facilitó la comprobación del funcionamiento de los botones y del joystick, así como la observación de los valores generados por el Arduino. Este mecanismo se empleó principalmente con fines de prueba y verificación, mientras que la comunicación USB HID constituye el medio principal de interacción del controlador con el sistema.
- **Representación en JSON:** Estándar de intercambio basado en texto plano estructurado mediante objetos clave-valor. Aunque su integración completa se mantiene en fase pendiente dentro del código fuente del repositorio, su propósito dentro de la arquitectura metodológica es permitir la deserialización dinámica de parámetros complejos directamente en los diccionarios de Godot.

IV-D. Desarrollo e integración de la experiencia interactiva

La aplicación desarrollada en Godot consiste en una experiencia interactiva de temática espacial en la que el usuario controla una nave y debe interactuar con diferentes asteroides. La comunicación entre el Arduino Pro Micro y el computador se realiza mediante USB HID, configurando el microcontrolador para que sea reconocido como un dispositivo de juego

(*gamepad*). De esta manera, los estados de los botones y los valores del joystick son enviados por el Arduino como entradas de un controlador y posteriormente son interpretados por el sistema de entrada de Godot mediante las acciones definidas en el *Input Map*. Estas entradas permiten controlar el movimiento de la nave y ejecutar acciones como disparar, impulsar y girar, estableciendo una conexión directa entre la manipulación del controlador físico y la respuesta dentro de la experiencia digital.

IV-E. Procedimiento de Medición

Para evaluar el desempeño de las implementaciones se estableció un procedimiento experimental basado en la captura de métricas de rendimiento:

1. **Latencia de transmisión:** Medición del tiempo transcurrido desde la pulsación o movimiento físico en el hardware hasta la respuesta visual reflejada en el simulador de vuelo en Godot.
2. **Legibilidad y depuración:** Evaluación cualitativa de la facilidad para diagnosticar errores mediante el monitor serial y la traza de eventos en la consola del motor.
3. **Costo de procesamiento:** Análisis del tamaño de los paquetes en bytes y su impacto en la sobrecarga del bucle principal (*main loop*) del simulador.

V. RESULTADOS

V-A. Título de subsección

Contenido de la subsección

V-A1. *Título de subsubsección:* Contenido de la subsubsección

VI. ANÁLISIS Y DISCUSIÓN

En este trabajo se presentó ..., se analizaron ..., y se demostró que Los resultados obtenidos permiten concluir que ..., lo cual abre la posibilidad de trabajos futuros relacionados con

VII. CONCLUSIÓN

En este trabajo se presentó ..., se analizaron ..., y se demostró que Los resultados obtenidos permiten concluir que ..., lo cual abre la posibilidad de trabajos futuros relacionados con

REFERENCIAS

- [1] Tanenbaum, A. S. y Wetherall, D. J. (2012). *Redes de computadoras* (5ª ed.). Pearson Educación. ISBN: 978-607-32-0817-8.
- [2] Stallings, W. (2004). *Comunicaciones y redes de computadores* (7ª ed.). Pearson Prentice Hall. ISBN: 84-205-4110-9.
- [3] Unity Technologies. (2024). *JSON Serialization*. Unity Documentation. Disponible en: <https://docs.unity3d.com/Manual/JSONSerialization.html>
- [4] Newton-King, J. (2024). *Json.NET Documentation*. Newtonsoft. Disponible en: <https://www.newtonsoft.com/json/help/html/Introduction.htm>
- [5] Godot Engine. (2024). *JSON class*. Godot Engine Documentation (Stable). Disponible en: https://docs.godotengine.org/en/stable/classes/class_json.html
- [6] Arduino. (2024). *Arduino Pro Micro Documentation*. Arduino Official Website. Disponible en: <https://www.arduino.cc/>
- [7] Botero, A. (2026). *Repositorio oficial del proyecto de Realidad Mixta: Experiencia Físico-Digital* [Código fuente]. GitHub. https://github.com/botero-dev/usbbog_realidadmixta_lab